NORTHWESTERN UNIVERSITY

Contract Checking as Effects

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

Master of Science

Field of Computer Science

By

Anh Khoa Nguyen

EVANSTON, ILLINOIS

December 2026

ABSTRACT

Contract Checking as Effects

Anh Khoa Nguyen

nice

Acknowledgements

Text for acknowledgments goes here. You can thank your advisors, peers, and family.

Preface

This is the preface (optional).

Dedication

This is the dedication (optional).

Table of Contents

List of Tables

List of Figures

CHAPTER 1

Introduction

Software contracts, or *behavioral contracts*, allow programmers to attach assertions to values, and functions. These assertions are runtime guarantees, on violation, an error is thrown, blaming the violating party. Contracts are most useful when placed on a function, enforcing the function's input and output values. While first-order functional contracts are useful, they are too simple. Higher-order functional contracts allows programmers to enforce contracts for higher-order functions. Dependent contracts are functional contracts allowing usage of the function argument during post-condition contract checking.

Recently, algebraic effect handlers has gained a large attraction, and is adopted in OCaml since version 5.0. Effect handlers open a new way to identify effects and handling them. Handlers are defined to handle one or many effects. A handler for an effect can stop the program or continue with some result. Handlers do not have state by default, but it can be implemented by applying the new states to the continuation closure, and let the underlying computation receives a state argument.

Combining the contract system and effect handlers is a new approach. Effectful Contract [10] demonstrates how both systems can be combined, providing contracts for effectful code. Contracts in their system remain a language feature. In this dissertation, instead of combining the effects and contracts, we ask ourselves whether contract checking itself can become an effect, and reuse the capabilities of handlers.

> *Contract checking can be encoded as effects, and delegate them to handlers can unify all contracts in the same context, while separated from main code.*

1.1 Contributions

This dissertation provides concrete evidence to the thesis claim. We model the two languages Contracts representing software contracts, and **Effects** representing effect handlers. Then a formal model for a compiler between Contracts and **Effects** is presented. This compiler is proven correct, and effect safe. Moreover, we provide an implementation of such compiler by providing a preprocessor to convert OCaml code annotated with contracts into run-able OCaml code based off this compiler model. Lastly, we provide a Redex PLT model to test our theorems.

Structure ...

CHAPTER 2

Formal Language Models

2.1 Base Language

We define Base language and its syntax in Figure 1. The language extends the PCF language with tuples and mutable cells. This will be the base language for both Contracts and `Effects`.

$\boxed{\textsf{Base}}$

$$
\begin{array}{rcl}
\tau & ::= & o \\
& \mid & \tau \to \tau \\
& \mid & \langle \tau, \tau \rangle \\
& \mid & \textsf{ref}(\tau) \\
& \mid & \textsf{unit} \\
\\
o & ::= & \text{num} \\
& \mid & \text{bool} \\
\\
e & ::= & n \mid b \mid \langle e, e \rangle \mid \text{x} \\
& \mid & \lambda x : \tau . e \mid \mu x : \tau . e \mid \textsf{loc} \\
& \mid & e + e \mid e - e \mid e \wedge e \mid e \vee e \\
& \mid & e\ e \mid \text{if } e \text{ then } e \text{ else } e \\
& \mid & \textsf{zero?}(e) \\
& \mid & \textsf{inj.l}(e) \mid \textsf{inj.r}(e) \\
& \mid & \textsf{new}(e) \mid \textsf{get}(e) \mid \textsf{set}(e, e)
\end{array}
$$

$$
\begin{array}{rcl}
n & ::= & 0 \mid -1 \mid 1 \mid \ldots \\
\\
b & ::= & \textsf{true} \mid \textsf{false} \\
\\
v & ::= & \textsf{unit} \\
& \mid & n \mid b \mid \langle v, v \rangle \mid \text{x} \mid \lambda x . e \mid \textsf{loc} \\
\\
E & ::= & \square \\
& \mid & E + e \mid v + E \mid E - e \mid v - E \\
& \mid & E \wedge e \mid v \wedge E \mid E \vee e \mid v \vee E \\
& \mid & E\ e \mid v\ E \mid \text{if } E \text{ then } e \text{ else } e \mid \textsf{zero?}(E) \\
& \mid & \langle E, e \rangle \mid \langle v, E \rangle, \textsf{inj.l}(E) \mid \textsf{inj.r}(E) \\
& \mid & \textsf{new}(E) \mid \textsf{get}(E) \mid \textsf{set}(E, e) \mid \textsf{set}(v, E)
\end{array}
$$

Figure 1: Base Language

$$\frac{}{\Gamma \vdash n : \mathsf{num}}\ \text{T-Num} \qquad \frac{}{\Gamma \vdash b : \mathsf{bool}}\ \text{T-Bool} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash \langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle}\ \text{T-Tuple}$$

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x : \tau_1 . e : \tau_1 \to \tau_2}\ \text{T-Lambda} \qquad \frac{\Gamma, x : \tau \vdash e : \tau}{\Gamma \vdash \mu x : \tau . e : \tau}\ \text{T-Fix}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : e_2}\ \text{T-App} \qquad \frac{\Gamma \vdash e_1 : \mathsf{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau}\ \text{T-If}$$

$$\frac{\Gamma \vdash e : \mathsf{num}}{\Gamma \vdash \mathsf{zero?}(e) : \mathsf{bool}}\ \text{T-Zero} \qquad \frac{\Gamma \vdash e : \langle \tau_1, \tau_2 \rangle}{\Gamma \vdash \mathsf{inj.l}(e) : \tau_1}\ \text{T-Inj-Left}$$

$$\frac{\Gamma \vdash e : \langle \tau_1, \tau_2 \rangle}{\Gamma \vdash \mathsf{inj.r}(e) : \tau_2}\ \text{T-Inj-Right} \qquad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathsf{new}(e) : \mathsf{ref}(\tau)}\ \text{T-New-Cell}$$

$$\frac{\Gamma \vdash e : \mathsf{ref}(\tau)}{\Gamma \vdash \mathsf{get}(e) : \tau}\ \text{T-Get-Cell} \qquad \frac{\Gamma \vdash e_1 : \mathsf{ref}(\tau) \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \mathsf{set}(e_1, e_2) : \mathsf{ref}(\tau)}\ \text{T-Get-Cell}$$

Figure 2: Type Rules for Base

$$\begin{array}{cccc}
\lambda x.e\,v & \longrightarrow & e[x := v] & \text{S-App} \\
\mu x.e & \longrightarrow & e[x := \mu x.e] & \text{SFix} \\
\text{if true then } e_1 \text{ else } e_2 & \longrightarrow & e_1 & \text{S-If-True} \\
\text{if false then } e_1 \text{ else } e_2 & \longrightarrow & e_2 & \text{S-If-False} \\
\mathsf{inj.l}(\langle v_1, v_2 \rangle) & \longrightarrow & v_1 & \text{S-Inj-Left} \\
\mathsf{inj.r}(\langle v_1, v_2 \rangle) & \longrightarrow & v_2 & \text{S-Inj-Right}
\end{array}$$

not really correct, maybe use different arrow

$$\frac{e_1 \longrightarrow e_2}{E[e_1], \sigma \longrightarrow E[e_2], \sigma}\ \text{Step}$$

$$\frac{\mathsf{loc} \notin \sigma}{E[\mathsf{new}(v)], \sigma \longrightarrow E[\mathsf{loc}], \sigma[\mathsf{loc} \rightarrow v]}\ \text{S-New-Cell} \qquad \frac{}{E[\mathsf{get}(\mathsf{loc})], \sigma \longrightarrow E[v], \sigma}\ \text{S-Get-Cell}$$

$$\frac{\mathsf{loc} \in \sigma}{E[\mathsf{set}(\mathsf{loc}, v)], \sigma \longrightarrow E[\mathsf{loc}], \sigma[\mathsf{loc} \rightarrow v]}\ \text{S-Set-Cell}$$

Figure 3: Reduction Rules for Base

2.2 Contracts Language

2.2.1 Example

Contracts provide a run-time validation of programs, similar to how assertions work. Contracts extend basic assertions to support functions, especially higher-order ones, correctness. Past research has move from pre/post condition placing on functions, into obligations [4], or monitors. These constructions support blame tracking, enabling the developer to know *who*, which function, is responsible for a violation.

Figure 4 illustrates a function protected with a monitor. This function should return a value that is even. When applied with 10, it is checked by the function's *domain contract*. Since the function does not impose any properties for its argument, the contract passes.

The body of the function is executed normally, but its result must pass the function's *range contract*. Because 11 is not an even number, the program terminates with a blame.

$$
\begin{array}{ll}
& \mathsf{mon}_j^{k,l}(\mathsf{flat}(\lambda x.\ \mathsf{true}) \to \mathsf{flat}(\mathsf{iseven}), \lambda x.x+1)\ 10 \\
\longrightarrow & \mathsf{guard}_j^{k,l}(\mathsf{flat}(\lambda x.\ \mathsf{true}) \to \mathsf{flat}(\mathsf{iseven}), \lambda x.x+1)\ 10 \\
\longrightarrow & \mathsf{mon}_j^{k,l}\big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{mon}_j^{l,k}(\mathsf{flat}(\lambda x.\ \mathsf{true}), 10)\big) \\
\longrightarrow & \mathsf{mon}_j^{k,l}\big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{guard}_j^{l,k}(\lambda x.\ \mathsf{true}, 10)\big) \\
\longrightarrow & \mathsf{mon}_j^{k,l}\big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{check}_j^{l}((\lambda x.\ \mathsf{true})\ 10, 10)\big) \\
\longrightarrow & \mathsf{mon}_j^{k,l}\big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{check}_j^{l}(\mathsf{true}, 10)\big) \\
\longrightarrow & \mathsf{mon}_j^{k,l}(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ 10) \\
\longrightarrow^* & \mathsf{mon}_j^{k,l}(\mathsf{flat}(\mathsf{iseven}), 11) \\
\longrightarrow^* & \mathsf{blame}_j^{k}
\end{array}
$$

Figure 4: Contracts in action

2.2.2 Formal

We define Contracts, in Figure 5, by extending the Base language with contracts. Contracts protect an expression, ensuring that the evaluated value conform to the contracts defined. Contracts are defined for all possible types of value. Base values, including booleans and numbers, are trivial to enfore a contract on them. While functions need to check for both its argument and return result. Not only that, functions are first-class citizen in the language, and we can form higher-order functions. A contract for function may want to “see” the argument when checking the result, such is the idea of dependent contracts. Both contracts higher-order functions, and dependent contracts semantics are discussed in [4].

The language supports contracts for tuples. Their contracts will be applied individually during execution. Mutable cells contracts follow the work of [2], where a special contract type $\mathsf{ref/c}(\kappa)$ is used to protect the cell's value.

$\boxed{\mathsf{Contracts}}$ extends Base

$$
\begin{array}{rcl@{\qquad}rcl}
\tau & ::= & \ldots & \kappa & ::= & \mathsf{flat}(e) \mid \kappa \to \kappa \mid \kappa \to_i \lambda x : \tau.\kappa \\
 & \mid & \mathsf{con}(\tau) & & \mid & \langle \kappa, \kappa \rangle \mid \mathsf{ref/c}(\kappa) \\
 \\
e & ::= & \ldots & v & ::= & \ldots \\
 & \mid & \mathsf{mon}_j^{k,l}(\kappa, e) & & \mid & \mathsf{guard}_j^{k,l}(\kappa, v) \\
 & \mid & \mathsf{check}_j^{k}(e, v) \\
 & \mid & \mathsf{blame}_p^{l} & E & ::= & \ldots \\
 & & & & \mid & \mathsf{mon}_j^{k,l}(\kappa, E) \\
 & & & & \mid & \mathsf{check}_j^{k}(E, v)
\end{array}
$$

Figure 5: Contracts language

$$\frac{\Gamma \vdash e : \tau \to \mathsf{bool}}{\Gamma \vdash \mathsf{flat}(e) : \mathsf{con}(\tau)}\ \text{Type-C-Flat} \qquad \frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \qquad \Gamma \vdash \kappa_2 : \mathsf{con}(\tau_2)}{\Gamma \vdash \kappa_1 \to \kappa_2 : \mathsf{con}(\tau_1 \to \tau_2)}\ \text{Type-C-Fun}$$

$$\frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \qquad \Gamma \vdash \lambda x : \tau_1.\kappa_2 : \tau_1 \to \mathsf{con}(\tau_2)}{\Gamma \vdash \kappa_1 \to_i \lambda x.\kappa_2 : \mathsf{con}(\tau_1 \to \tau_2)}\ \text{Type-C-DepFun}$$

$$\frac{\Gamma \vdash \kappa : \mathsf{con}(\tau)}{\Gamma \vdash \mathsf{ref/c}(\kappa) : \mathsf{con}(\mathsf{ref}(\tau))}\ \text{Type-C-Ref}$$

$$\frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \qquad \Gamma \vdash \kappa_2 : \mathsf{con}(\tau_2)}{\Gamma \vdash \langle \kappa_1, \kappa_2 \rangle : \mathsf{con}(\langle \tau_1, \tau_2 \rangle)}\ \text{Type-C-Tuple}$$

$$\frac{\Gamma \vdash \kappa : \mathsf{con}(\tau) \qquad \Gamma \vdash e : \tau}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e) : \tau}\ \text{Type-C-Mon} \qquad \frac{}{\Gamma \vdash \mathsf{blame}_l^{k} : \tau}\ \text{Type-C-Blame}$$

Figure 6: Type Rules for Contracts

When a contract violation occurs, a blame is thrown with the party, caller or callee. The semantics for correct blaming has been studied under [2] and [3]. Following previous works, We use *indy* semantics for dependent contracts. We use $\mathsf{guard}_j^{k,l}(\kappa, v)$ as the run-time proxy to value v procted under a contract κ. Guard is a value, the semantics adapt to usage of guards, and expand further, sometimes back to monitor to evaluate inner expressions. A guard on a basic value will trigger a $\mathsf{check}_j^k(e\ v, v)$ and returns the value v if it passes the predicate e, or returns a blame_j^k to party k with contract location j. The evaluation rules for Contracts is presented in Figure 7.

$$
\begin{array}{rcll}
\mathsf{check}_j^k(\mathrm{tt}, v) & \longrightarrow & v & \text{Check-True} \\
\mathsf{check}_j^k(\mathrm{ff}, v) & \longrightarrow & \mathsf{blame}_j^k & \text{Check-False} \\
\mathsf{mon}_j^{k,l}(\kappa, v) & \longrightarrow & \mathsf{guard}_j^{k,l}(\kappa, v) & \text{Mon-Guard} \\
\mathsf{guard}_j^{k,l}(v, \mathsf{flat}(e)) & \longrightarrow & \mathsf{check}_j^k(e\ v, v) & \text{Guard-Flat} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \rightarrow \kappa_2, v_1)\ v_2 & \longrightarrow & \mathsf{mon}_j^{k,l}\big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big) & \text{Guard-Func} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \rightarrow_i \lambda x.\kappa_2, v_1)\ v_2 & \longrightarrow & \mathsf{mon}_j^{k,l}\big(\kappa_2', v_1\ \mathsf{mon}_j^{k,l}(\kappa_1, v_2)\big) & \text{Guard-Dep} \\
 & & \multicolumn{2}{r}{\text{where } \kappa_2' = \big\{\mathsf{mon}_j^{l,j}(\kappa_1, v_2)/x\big\}\kappa_2} \\
\mathsf{inj.l}\big(\mathsf{guard}_j^{k,l}(\langle \kappa_1, \kappa_2\rangle, \langle v_1, v_2\rangle)\big) & \longrightarrow & \mathsf{guard}_j^{k,l}(\kappa_1, v_1) & \text{Guard-Inj-Left} \\
\mathsf{inj.r}\big(\mathsf{guard}_j^{k,l}(\langle \kappa_1, \kappa_2\rangle, \langle v_1, v_2\rangle)\big) & \longrightarrow & \mathsf{guard}_j^{k,l}(\kappa_2, v_2) & \text{Guard-Inj-Right} \\
\mathsf{get}\big(\mathsf{guard}_j^{k,l}(\kappa, v)\big) & \longrightarrow & \mathsf{mon}_j^{k,l}(\kappa, \mathsf{get}(v)) & \text{Mon-Get} \\
\mathsf{set}\big(\mathsf{guard}_j^{k,l}(\kappa, v_1), v_2\big) & \longrightarrow & \mathsf{mon}_j^{k,l}(\mathsf{ref/c}(\kappa), e) & \text{Mon-Set} \\
 & & \multicolumn{2}{r}{\text{where } e = \mathsf{set}\big(v_1, \mathsf{mon}_j^{l,k}(\kappa, v_2)\big)}
\end{array}
$$

$$
\frac{E \neq \square}{E\big[\mathsf{blame}_j^k\big] \Longrightarrow \mathsf{blame}_j^k}\ \text{Blame}
$$

Figure 7: Reduction Rules for Contracts

2.3 `Effects` Language

2.3.1 Example

Effect handler is a new construct similar to how try/catch was designed. In fact, the idea is similar to resumable exception. An effect is invoked (throw), and a handler (try) for the effect installed prior catches. A continuation is pushed into the catch context, and can be invoked to continue the execution, or abort. Yapping...

In Figure 8, we lay out a simple example. A program wants to compute $(\mathscr{D}\ 2) + 1$ is registered with a handler h for effect $\mathscr{D}$, basically *duplicating* its input. When an effect is perform, the main program pauses the execution, and push everything later into a continuation $k = \lambda y.\ \texttt{handle}\ h\ (y + 1)$, at (i). Here, the continuation register the handler h again, in case the rest of the program uses $\mathscr{D}$. When the program is a value, the handler removes itself (ii).

$$
\begin{array}{lll}
h & = & \{\mathscr{D} \to \lambda v.\lambda k.k\ (v + v)\}
\end{array}
$$

$$
\begin{array}{lll}
 & \texttt{handle}\ h\ ((\texttt{perform}\ \mathscr{D}\ \mathsf{num}\ 2) + 1) & \\
\Longrightarrow & \texttt{handle}\ h\ \ \texttt{perform}\ \mathscr{D}\ \mathsf{num}\ 2 & \\
\Longrightarrow & (\lambda \underline{v}.\lambda \underline{k}.k\ (v + v))\ \underline{2}\ \underline{(\lambda y.\ \texttt{handle}\ h\ (y + 1))} & \text{(i)} \\
\Longrightarrow^* & (\lambda y.\ \texttt{handle}\ h\ (y + 1))\ (2 + 2) & \\
\Longrightarrow^* & \texttt{handle}\ h\ 5 & \text{(ii)} \\
\Longrightarrow & 5 &
\end{array}
$$

Figure 8: Handler in action

Handlers can encode states. This is a nice feature to provide a common state across the code. We illustrate the example in Figure 9, where a simple state can be managed by two effects *get* $\mathscr{G}$, and *set* $\mathscr{S}$. As can be seen, the continuation now returns a function expecting a state. During handler execution, this state is applied automatically if the handler is installed with initial state. The continuation resumes the program with the new state, made possible because handler is installed with new state.

In the example, the execution enter a stuck state if we just remove the handler. There are many ways to resolve this problem. The first approach requires the program awareness of the state and return a function taking the state, ignore the state, and return the result. The second approach is to use *parameterized handlers* [7], which puts the state into the handler expression. The third approach is to augment the handler with a special effect `return`, which modifies the value before removing the handler.

$$
\begin{array}{lcl}
h & = & \{\mathscr{G} \to \lambda v.\lambda k.\lambda s.k\ s\ s; \mathscr{S} \to \lambda v.\lambda k.\lambda s.k\ s\ v; \texttt{return} \to \lambda x.\lambda s.x\} \\
 & & (\texttt{handle}\ h\ ((\texttt{perform}\ \mathscr{S}\ \mathsf{num}\ 42) + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit})))\ 0 \\
\Longrightarrow^* & & (\lambda \underline{v}.\lambda \underline{k}.\lambda \underline{s}.k\ s\ v)\ \underline{42}\ \underline{\lambda y.\ \texttt{handle}\ h\ (y + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}))}\ \underline{0} \\
\Longrightarrow^* & & (\lambda y.\ \texttt{handle}\ h\ (y + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit})))\ 0\ 42 \\
\Longrightarrow^* & & \texttt{handle}\ h\ (0 + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}))\ 42 \\
\Longrightarrow^* & & \texttt{handle}\ h\ 42\ 42 \\
\Longrightarrow^* & & (\lambda x.\lambda s.x)\ 42\ 42 \\
\Longrightarrow^* & & 42
\end{array}
$$

Figure 9: Handler with state

2.3.2 Formalism

We continue to extend the Base language with effects and effect handlers in Listing 1. This is done by typing the effects separately from the base types. This practice has been introduced to formalize the Koka[1] language. The approach we take here resembles the System F^{ε} language introduced in [15]. We added polymorphism through type application, and kinds to Base language. Two special kinds are added to represent effects (**eff**) and effect labels (**lab**). A label $l \in \texttt{lab}$ is a group of operations. And an effect $\varepsilon \in \texttt{eff}$ is composed by chaining **lab** together with other effects. An empty effect $\langle\rangle$ defines no labels, therefore no effects can be performed. In otherwords, **eff** defines the set of effects that can be performed. A function can now produce some effects $\varepsilon \in \texttt{eff}$, and is represented in its type $\tau_1 \to^{\varepsilon} \tau_2$. Original type for function $\tau_1 \to \tau_2$ is understood as pure, without effects, $\tau_1 \to^{\langle\rangle} \tau_2$.

The language is then extended with handlers, handler installation ($\texttt{handle}\ h\ e$), and calling/performing an effect ($\texttt{perform}\ \texttt{op}\ \tau$). Handlers are a set of operations distinguishable by the operation's name. A handler denotes a single effect usually by its label $l \in \texttt{lab}$. Performing an effect is analogous function application, therefore $\texttt{perform}\ \texttt{op}\ \tau$ is treated as a value, where application for it has a distinct reduction rule. The evaluation context is thus extended to support handler installation.

[1]https://koka-lang.github.io/

$\boxed{\texttt{Effects}}$ extends Base

$$
\begin{array}{rcl}
\tau & ::= & \ldots \\
 & | & \alpha^k \mid c^k\tau\ldots \mid \tau \to^{\varepsilon} \tau \mid \forall\alpha^k.\tau \\
 & & \\
k & ::= & * \\
 & | & k \to k \\
 & | & \texttt{eff} \\
 & | & \texttt{lab} \\
 & | & \texttt{blab} \\
 & & \\
e & ::= & \ldots \\
 & | & e[\tau] \\
 & | & \texttt{handle}\ h\ e \\
 & | & \texttt{blame}_p \\
 & & \\
h & ::= & \{\texttt{return} \to \lambda x.e, \texttt{op} \to f, \ldots, \}
\end{array}
\qquad
\begin{array}{rcl}
v & ::= & \ldots \\
 & | & \Lambda\alpha^k.v \\
 & | & \texttt{handler}\ h \\
 & | & \texttt{perform}\ \texttt{op}\ \tau \\
 & & \\
E & ::= & \ldots \\
 & | & E[\tau] \\
 & | & \texttt{handle}\ h\ E \\
 & & \\
F & ::= & \square \\
 & | & F + e \mid v + F \mid F - e \mid v - F \\
 & | & F \wedge e \mid v \wedge F \mid F \vee e \mid v \vee F \\
 & | & F\ e \mid v\ F \\
 & | & \text{if } F \text{ then } e \text{ else } e \\
 & | & F[\tau]
\end{array}
$$

Listing 1: `Effects` language

Typing for an effect language requires some consideration. Following the works in [15], we define the type judgement relationship $\Delta \vdash e : \tau \mid \varepsilon$ between a type context Δ, an expression e and assert it against type τ with some possible effects ε. Variables and base values do not have effects, they can take any effects. For this reason, we define $\Delta \vdash_{\text{val}} v : \tau$. Handlers are typed using $\Delta \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$, that it handles a *single* effect l, other effects in ε remains unhandled, and all operations returns τ. All type rules unique to `Effects` is presented in Figure 10.

$$\frac{\ldots}{\ldots}\ \text{SomeRule}$$

Figure 10: Type Rules for `Effects`

Lastly the semantic model of `Effects` is presented in Listing 2. A handler can be installed to a function as a shorthand, where it applies the function with a unit argument, which to be ignored. The reduction for handling an effect calling is encoded as *deep handler*.

Supporting the shallow should be straightforward by not installing the handler again in the continuation k. There is little advantage to support shallow handlers, therefore, only deep handlers are used. While looking for the handler, we make sure we get the innermost handler that can handle the operation.

$$\begin{array}{lcll} (\Lambda\alpha^k.v)[\tau] & \Longrightarrow & v[\alpha := \tau] & \text{Step-E-TApp} \\ \texttt{handler}\ h\ v & \Longrightarrow & \texttt{handle}\ h\ (v\ ()) & \text{Step-E-Handler} \\ \texttt{handle}\ h\ v & \Longrightarrow & v & \text{Step-E-Return} \end{array}$$

$$\frac{\texttt{op} \notin \texttt{bop}(E) \wedge (\texttt{op} \to f) \in h \qquad \texttt{op} : \forall\alpha.\tau_1 \to \tau_2 \in \Sigma(l) \quad k = \lambda x : \tau_2[\alpha := \tau].\ \texttt{handle}\ h\ E[x]}{\texttt{handle}\ h\ E[\texttt{perform}\ \texttt{op}\ \tau\ v] \Longrightarrow f[\tau]\ v\ k}\ \text{Step-E-Perform}$$

$$\frac{E \neq \square}{E\left[\texttt{blame}_p\right] \Longrightarrow \texttt{blame}_p}\ \text{Effect-Error}$$

Helper functions

$$\begin{array}{ccc} \texttt{bop}(\square) & = & \varnothing \\ \texttt{bop}(E\ e) & = & \texttt{bop}(E) \\ \vdots & \vdots & \vdots \\ \texttt{bop}(\texttt{handle}\ h\ E) & = & \texttt{bop}(E) \cup \{\texttt{op} \mid (\texttt{op} \to f) \in h\} \end{array}$$

Listing 2: Reduction Rules for `Effects`

CHAPTER 3

Formal Compiler

A compiler from Contracts to **Effects** is straightforward for expressions inherited from the Base language. The compiler keeps these same expressions, albeit each sub-expression must be compiled *recursively* to ensure no contracts remains after compilation. Remaining expressions are contract-related expressions. Our idea is that a contract check is a side effect, defined as $\mathscr{C}$. This effect idealy receives a tuple $\langle l, \langle e, v\rangle\rangle$ of blame label l, contract predicate e, and to-be-checked value v. Then, a flat contract is simly an effect perform with its correct inputs.

Functional contracts are more complex. Because the effect can only be invoked on a flat contract, the compiler recursively expands the expression, pads the expression into a function when it is expecting a function. During the compilation, new variables are generated to pad the expression into a function. The process is also applicable to dependent contracts. However, the dependent contract needs to update the post contract with a contract for argument. This is trivial but requires a duplication of compilation with a different label. The rules are crafted carefully to prevent syntaxs of **Effects** into Contracts while compiling the dependent contracts. The compiled argument contract is delayed substitution, using a random variable as placeholder, until we have the **Effects** expression which we can safely replace into.

Compilation for tuple contracts are straightforward. Each tuple entries are compiled individualy, and reconstructed into a new tuple. Then, each entries are protected, flat contracts are checked immediately during tuple construction; functions are protected and should be checked when used.

3.1 Handler design

After all flat contracts are converted into performing an effect, we need a handler. A simple handler can be installed like below:

$$h_{\text{naive}} = \{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \text{if } e\, v \text{ then } k\, v \text{ else } \texttt{blame}_l\}$$

$$\text{where } \lambda\langle l, \langle e, v\rangle\rangle \text{ deconstructs the value into tuple pattern}$$

This handler performs the check for the value v, with the predicate/contract e. The continuation k is invoked with the value if the check passes, and blame the label l if the contract is violated. At first glance, this handler is correct. In fact, the handler should produce correct semantic, as long as no dependent contracts are used. When dependent contracts are used, e is now populated with monitored arguments, which after compilation should include $\texttt{perform}\ \mathscr{C}\ \tau\ \text{arg}$. And since the handler is moved into the continuation k, there is no handler at $e\, v$. Take the following program wrapped with the naive handler h_{naive}.

$$
\begin{array}{ll}
 & \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1) \to_i \lambda y.\ \mathsf{flat}(e_2), f)\ 42 \\
 & \qquad \text{where } e_1 = \lambda x.\ \mathsf{true},\ e_2 = \lambda x. x > y, \text{ and } f = \lambda x. x + x \\
\rightarrowtail & (\lambda x_1.\ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle)\ 42 \\
 & \qquad \text{where } e_3 = \{(\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)/y\}e_2
\end{array}
$$

$$
\begin{array}{ll}
 & \texttt{handler}\ h_{\text{naive}}\ \lambda\_.(\lambda x_1.\ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle\ 42) \\
\Longrightarrow & \texttt{handle}\ h_{\text{naive}}\ (\lambda x_1.\ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle\ 42) \\
\Longrightarrow & \texttt{handle}\ h_{\text{naive}}\ \ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle)\rangle\rangle \\
 & \qquad \text{where } e_3' = \{(\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle)/y\}e_2 \\
\Longrightarrow^* & \texttt{handle}\ h_{\text{naive}}\ \ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', f\ 42\rangle\rangle \\
\Longrightarrow^* & \texttt{handle}\ h_{\text{naive}}\ \ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', 84\rangle\rangle \\
\Longrightarrow^* & \text{if } e_3'\ 84 \text{ then } (\lambda x.\ \texttt{handle}\ h_{\text{naive}}\ x)\ 84 \text{ else } \texttt{blame}_k \\
\Longrightarrow^* & \text{if } 84 > (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle) \text{ then } (\lambda x.\ \texttt{handle}\ h_{\text{naive}}\ x)\ 84 \text{ else } \texttt{blame}_k \\
 & \textit{stuck}
\end{array}
$$

To resolve this issue, we define a recursive function wrap that will reinstall the handler at $e\ v$ making sure all effects during contract checking is handled. All compilation rules are defined in Figure 11. With wrap defined, we can make some theorems about the compiler.

The compiler should produce same type expression. However, this is problematic when effects are introduced. Recall that the type system separates between pure function, and effectful functions. And type judgement separates between non-effectful computations and effectful computations. Depending on the source, compiled program may have the same type with no effects (no monitors), or same type with some effects (monitors during computation), or purely produce a function wrapped by monitors, or effectfully produce a function wrapped by monitors (monitors during computation).

Regardless, wrapping the compiled program with wrap should handle all possible effects.

Theorem 3.1. Compiler Type Preservation.

If $\cdot \vdash e_1 : \tau \rightarrowtail e_2$ then either

- $\cdot \vdash e_2 : \tau \mid \langle\rangle$ or
- $\tau = \tau_1 \to \tau_2$ and $\cdot \vdash e_2 : \tau_1 \to^{\mathscr{C}} \tau_2 \mid \langle\rangle$ or
- $\cdot \vdash e_2 : \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ or
- $\tau = \tau_1 \to \tau_2$ and $\cdot \vdash e_2 : \tau_1 \to^{\mathscr{C}} \tau_2 \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$.

Theorem 3.2. Compiler Effect Safety with wrap.

If $\cdot \vdash e_1 : \tau \rightarrowtail e_2$ then either

- $\cdot \vdash \mathsf{wrap}[\tau]\ (\lambda\_ : \mathsf{unit}.e) : \tau \mid \langle\rangle$.
- $\tau = \tau_1 \to \tau_2\ \cdot \vdash \mathsf{wrap}[\tau]\ (\lambda\_ : \mathsf{unit}.e) : \tau_1 \to^{\mathscr{C}} \tau_2 \mid \langle\rangle$.

$$\frac{}{\Gamma \vdash \textsf{mon}_j^{k,l}(\textsf{flat}(e), v) : \tau \rightarrowtail \texttt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e, v\rangle\rangle}\ \text{Compile-Flat}$$

$$\frac{x \notin \Gamma \quad \Gamma \vdash \textsf{mon}_j^{l,k}(\kappa_1, x) : \tau_1 \rightarrowtail x_1 \quad \Gamma, x : \tau_1 \vdash \textsf{mon}_j^{k,l}(\kappa_2, e\ x_1) : \tau_2 \rightarrowtail e_1}{\Gamma \vdash \textsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) : \tau_1 \to \tau_2 \rightarrowtail \lambda x : \tau_1 . e_1}\ \text{Compile-Func}$$

$$\frac{\begin{array}{c} x, x_\kappa \notin \Gamma \quad \Gamma \vdash \textsf{mon}_j^{l,k}(\kappa_1, x) : \tau_1 \rightarrowtail x_1 \quad \Gamma \vdash \textsf{mon}_j^{l,j}(\kappa_1, x) : \tau_1 \rightarrowtail x_2 \\ \kappa_3 = \{x_\kappa / x\}\kappa_2 \quad \Gamma, x : \tau_1 \vdash \textsf{mon}_j^{k,l}(\kappa_3, e\ x_1) : \tau_2 \rightarrowtail e_1 \quad e_2 = \{x_2 / x_\kappa\} e_1 \end{array}}{\Gamma \vdash \textsf{mon}_j^{k,l}\left(\kappa_1 \xrightarrow{d} \lambda x . \kappa_2, e\right) : \tau_1 \to \tau_2 \rightarrowtail \lambda x : \tau_1 . e_2}\ \text{Compile-Dep}$$

$$\frac{\Gamma \vdash \textsf{mon}_j^{k,l}(\kappa_1, v_1) : \tau \rightarrowtail e_3 \quad \Gamma \vdash \textsf{mon}_j^{k,l}(\kappa_2, v_2) : \tau \rightarrowtail e_4}{\Gamma \vdash \textsf{mon}_j^{k,l}(\langle \kappa_1, \kappa_2\rangle, \langle v_1, v_2\rangle) : \tau \rightarrowtail \langle e_3, e_4\rangle}\ \text{Compile-Tuple}$$

$$\begin{array}{lcl} \texttt{h}_{\mathscr{C}} : \texttt{lab} & = & \{\mathscr{C} \to \forall \alpha . \langle \texttt{blab}, \langle \alpha \to \textsf{bool}, \alpha\rangle\rangle \to \alpha\} \\ \textsf{wrap} & = & \mu w : \forall \alpha . ((() \to^{\langle \texttt{h}_{\mathscr{C}}\rangle} \alpha) \to \alpha) . \Lambda \alpha .\ \lambda f : (() \to^{\langle \texttt{h}_{\mathscr{C}}\rangle} \alpha) . \\ & & \texttt{handler}\ \{\mathscr{C} \to \lambda \langle l, \langle e, v\rangle\rangle . \lambda k .\ \text{if}\ w[\textsf{bool}]\ (\lambda \_ . e\ v)\ \text{then}\ k\ v\ \text{else}\ \texttt{blame}_l\}\ f \\ & & \qquad \text{where}\ \lambda\langle l, \langle e, v\rangle\rangle\ \text{deconstructs the argument as tuple pattern} \end{array}$$

Figure 11: Compiler Rules

3.2 Compiling Contracts for Mutable Cells

3.2.1 Observation

Contracts uses $\textsf{guard}_j^{k,l}(\kappa, v)$ as a proxy to mutable cells. Get and set operation go through this guard to enfore contracts at run-time. This proxy makes the action oblivious whether it is a real cell or a guarded value. Consider this simple function that calls $\textsf{get}(c)$ for any cell.

$$\lambda c.\ \textsf{get}(c)$$

This function should work with both cells $\textsf{new}(v)$ and guards constructed from $\textsf{mon}_j^{k,l}(\textsf{ref/c}(e), l)$ (where l a location). **Effects** does not have special reduction rules for mutable cells, therefore, we need a unified interface between cells and protected cells, and rewrite cell operations to use that unified interface.

3.2.2 Unified Interface

Our compiler has supports for functions and tuples, this is useful for us to design such unified interface. A mutable cell must support get and set, we can encode this as a tuple of 2 functions. Contracts on a cell is now converted into contracts placed on these get and set functions. Cell's location is enclosed inside the closure, and should be accessible when invoked.

$$\begin{aligned}
\texttt{newcell} &= \lambda v.(\lambda l.\langle \lambda_.\ \mathsf{get}(l), \lambda v.\ \mathsf{set}(l, v)\rangle)\ \mathsf{new}(v) \\
\texttt{getcell} &= \lambda b.\ \mathsf{inj.l}(b)\ \mathsf{unit} \\
\texttt{setcell} &= \lambda b.\lambda v.(\lambda_.b\ (\mathsf{inj.r}(b)\ v))
\end{aligned}$$

Listing 3: Mutable Cells Wrappers

The compiler should first convert all cell operations into this new wrapper. Then convert all $\mathsf{ref/c}(\kappa)$ into tuple contracts for get and set functions.

$$\mathsf{mon}_j^{k,l}(\mathsf{ref/c}(\kappa), e) \rightarrowtail \mathsf{mon}_j^{k,l}(\langle \text{any} \to \kappa, \text{any} \to \kappa \to \text{any}\rangle, e)$$

CHAPTER 4

Implementation

We implement the formal compiler model into OCaml, by adding a preprocessor. We utilize the OCaml attribute to add contract annotations to variables. The preprocessor rewrites annotated variables according to the compiler rules defined. Note that the compiler does not place a handler during compilation, but after the compilation. Because the handler must be installed around the whole program, more precisely, around everywhere the contract annotated variables are used. The developer can install the handler manually by invoking a predefined wrapper, or annotate a function as main and the preprocessor rewrites to add wrapper automatically.

Annotations are OCaml annotations. Although OCaml annotations support string-like payload, we find it better to use a OCaml type expression as payload to define contracts. Annotations are not code, therefore we cannot support first-order contracts, and use function names as predicates. In Listing 4, we define the contract annotation syntax that developers can use to annotate their OCaml code with contracts.

```ocaml
(* flat contract *)
let [@contract : predicate] x : t = e

(* function contract
 * arguments are inside a tuple
 *)
let [@contract : (p1_pred * p2_pred) -> f_pred]
  f (p1 : t1) (p2 : t2) : t = f_body

(* dependent function contract
 * similarly to function contract, annotated with as 'dep
 *)
let [@contract : (p1_pred * p2_pred) -> g_pred as 'dep]
  g (p1 : t1) (p2 : t2) : t = g_body

(* tuple contract
 * using tuple syntax and annotated with as 'tuple
 *)
let [@contract : (t1_pred * t2_pred) as 'tuple]
  tup : (t1 * t2) = (e, e)
```

Listing 4: OCaml Contract Annotation Example

The preprocessor rewrite using the compiler rules. All flat contracts are compiled into a single effect, formally it is $\mathscr{C}$, implementation wise we define a name `Contract.Check`.

```ocaml
module Contract = struct
type _ Effect.t +=
  | Check : (string * ('a -> bool) * 'a) -> 'a Effect.t
end
```

wrap is defined similarly in OCaml, we remove the type annotations for clarity.

```ocaml
let rec wrap thunk =
  Effect.Deep.match_with thunk ()
  {
    Effect.Deep.retc = (fun x -> x);
    exnc = (fun e -> raise e);
    effc = fun eff ->
      let open Effect.Deep in
      match eff with
```

```
      | Flat (label, check, arg) ->
          Some (fun k ->
            let ok = wrap (fun () -> check arg) in
            if ok
            then continue k arg
            else discontinue k (Blame label))
      | _ -> None
  }
```

4.1 Contract States and Interaction

4.1.1 Global State

Because the entire program is wrapped into a single handler for all effects, we can unify a single state for effects only. Predicates must now receive an extra argument, which is the context. The wrap function is now defined with an extra argument current context, and it returns the result and updated context. At the callsite, it is provided with the initial value. States are provided to the handler, therefore the handler provide two effects to get and set the state.

```ocaml
type state = int
let state_init = 0

let rec wrap_inner : type r. (unit -> r) -> state -> r * state =
  fun thunk init ->
  match_with thunk ()
  {
    retc = (fun x state -> (x, state));
    exnc = (fun e _state -> raise e);
    effc = fun (type a) (eff : a Effect.t) ->
      let handle_eff : ((a, state -> r * state) continuation -> state -> r * state) option =
      match eff with
      | Get -> Some (fun k state -> continue k state state)
      | Set x -> Some (fun k _state -> continue k () x)
      | Flat (label, check, arg) ->
```

```
          Some (fun k state ->
            let (ok, updated_state) = wrap_inner (fun () -> check arg) state in
            if ok
            then continue k arg updated_state
            else discontinue k (Blame label) updated_state)

      | _ -> None
    in
    handle_eff
  }
  init

let wrap main =
  let r, _ = wrap_inner main state_init in
  r

let () = wrap main
```

This approach works, but with a small caveat. Main code can access this state, and can potentially modifies the contract's state. This issue is easily resolved by making two more handlers, one handler for state, and one handler perform recursive wrapping for contract checking code. The state handler is installed in the main wrapper, placing on the top most contract checking code. When a contract check occurs it opens a new scope, starting with the current state. All dependent contract checking code originates from this top most contract update on the same state. Once the top most contract code returns, the state is updated at the continuation of the main wrapper. Because the state is registered by the main wrapper, and the handler for state is registered only when contract checking code executes, main code has no access to this state.

However, in this implementation, a `Flat` effect might be invoked arbitrarily by the main code. In an actual language implementation, we suggest this effect be private, or a special keyword to prevent developers interact with the effect state.

```ocaml
let rec with_state : type res. (unit -> res) -> int -> res * int =
  fun thunk init ->
  match_with thunk ()
  { retc = (fun x s -> (x, s));
    exnc = (fun e _ -> raise e);
    effc = fun (type a) (eff : a Effect.t) ->
      let handler : ((a, int -> res * int) continuation -> int -> res * int) option =
        match eff with
        | Get -> Some (fun k s -> continue k s s)
        | Set x -> Some (fun k _ -> continue k () x)
        | _ -> None
      in handler
  } init

let rec wrap_inner : type res. (unit -> res) -> res =
  fun thunk ->
  match_with thunk ()
  { retc = (fun x -> x);
    exnc = (fun e -> raise e);
    effc = fun (type a) (eff : a Effect.t) ->
      match eff with
      | Flat (label, check, arg) ->
        Some (fun k ->
          let ok = wrap_inner (fun () -> check arg) in
          if ok then continue k arg
          else discontinue k (Blame label))
      | _ -> None
  }

let rec wrap : type res. (unit -> res) -> int -> res * int =
  fun thunk init ->
  match_with thunk ()
  { retc = (fun x s -> (x, s));
    exnc = (fun e _ -> raise e);
    effc = fun (type a) (eff : a Effect.t) ->
      let handler : ((a, int -> res * int) continuation -> int -> res * int) option =
```

```
      match eff with
      | Flat (label, check, arg) ->
          Some (fun k current_state ->
            let (ok, next_state) =
              with_state (fun () -> wrap_inner (fun () -> check arg)) current_state
            in
            if ok then continue k arg next_state
            else discontinue k (Blame label) next_state)
      | _ -> None
    in handler
} init
```

4.2 Contract Interaction

All contract are unified under a single context. This context is accessible through two effects `Get` and `Set`. Let this context be a simple key-value storage, then programmers can devise complicated schemes to track function calls, current function scope, internal contract state, etc, ...

Bibliography

[1] Andrej Bauer and Matija Pretnar. 2014. An Effect System for Algebraic Effects and Handlers. *Log. Methods Comput. Sci.* 10, 4 (2014). https://doi.org/10.2168/LMCS-10(4:9)2014

[2] Christos Dimoulas and Matthias Felleisen. 2011. On contract satisfaction in a higher-order world. *ACM Trans. Program. Lang. Syst.* 33, 5 (2011), 16:1–16:29. https://doi.org/10.1145/2039346.2039348

[3] Christos Dimoulas, Sam Tobin-Hochstadt, and Matthias Felleisen. 2012. Complete Monitors for Behavioral Contracts. In *Programming Languages and Systems - 21st European Symposium on Programming, ESOP 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings (Lecture Notes in Computer Science)*, 2012. Springer, 214–233. https://doi.org/10.1007/978-3-642-28869-2_11

[4] Robert Bruce Findler and Matthias Felleisen. 2013. ICFP 2002: Contracts for higher-order functions. *ACM SIGPLAN Notices* 48, 4S (2013), 34–45. https://doi.org/10.1145/2502508.2502521

[5] Oleg Kiselyov and KC Sivaramakrishnan. 2018. Eff directly in OCaml. *arXiv preprint arXiv:1812.11664* (2018).

[6] Daan Leijen. 2016. *Algebraic Effects for Functional Programming.* Retrieved from https://www.microsoft.com/en-us/research/publication/algebraic-effects-for-functional-programming/

[7] Daan Leijen. 2017. Type directed compilation of row-typed algebraic effects. *SIGPLAN Not.* 52, 1 (January 2017), 486–499. https://doi.org/10.1145/3093333.3009872

[8] Paul Blain Levy. 2013. Call-by-push-value. Doctoral dissertation.

[9] Bertrand Meyer. 1992. The eiffel programming language. *See http://www. eiffel. com* (1992).

[10] Cameron Moy, Christos Dimoulas, and Matthias Felleisen. 2024. Effectful Software Contracts. *Proc. ACM Program. Lang.* 8, POPL (2024), 2639–2666. https://doi.org/10.1145/3632930

[11] Gordon D. Plotkin and John Power. 2003. Algebraic Operations and Generic Effects. *Appl. Categorical Struct.* 11, 1 (2003), 69–94. https://doi.org/10.1023/A:1023064908962

[12] Matija Pretnar. 2010. Logic and handling of algebraic effects. Doctoral dissertation. Retrieved from https://hdl.handle.net/1842/4611

[13] Matija Pretnar. 2015. An Introduction to Algebraic Effects and Handlers. Invited tutorial paper. In *The 31st Conference on the Mathematical Foundations of Programming Semantics, MFPS 2015, Nijmegen, The Netherlands, June 22-25, 2015 (Electronic Notes in Theoretical Computer Science)*, 2015. Elsevier, 19–35. https://doi.org/10.1016/J.ENTCS.2015.12.003

[14] K. C. Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. 2021. Retrofitting Effect Handlers onto OCaml. *CoRR* (2021). Retrieved from https://arxiv.org/abs/2104.00250

[15] Ningning Xie, Jonathan Immanuel Brachthäuser, Daniel Hillerström, Philipp Schuster, and Daan Leijen. 2020. Effect handlers, evidently. *Proc. ACM Program. Lang.* 4, ICFP (2020), 99:1–99:29. https://doi.org/10.1145/3408981

[16] Dana N Xu. 2014. Dynamic Contract Checking for OCaml.

CHAPTER A

Contract Erasure

CHAPTER B

Effects Safety

CHAPTER C

Compiler Soundness

Proof of Theorem 1. something Lemma 3 □

Proof of Theorem 2. something else □

Lemma 3.1.